

บทที่ 2

Extensible Markup Language (XML)

2.1 บทนำ

XML ถูกกำหนดโดยกลุ่มทำงาน XML ของสถาบัน World Wide Web Consortium (W3C) กลุ่มทำงานกลุ่มนี้ได้บรรยายถึงภาษา XML ไว้ว่า “Extensible Markup Language (XML) เป็นเซตย่อยของ Standard Generalized Markup Language (SGML) จุดมุ่งหมายคือให้สามารถใช้ SGML ในการบริการรับข้อมูลและประมวลผลบนเครือข่ายอินเทอร์เน็ตซึ่งปัจจุบันสามารถทำได้ด้วยภาษา HTML เท่านั้น XML ถูกออกแบบมาเพื่อให้เป็นวิธีการที่ง่ายและสามารถประมวลผลร่วมกันทั้ง SGML และ HTML”

เนื่องจากอิลิเมนต์ของ XML ไม่มีความหมายจริงๆ จึงเป็นสาเหตุที่ทำให้ XML เป็นภาษาที่กำลังมาเป็นภาษาที่มีความสำคัญ เพราะ XML เป็นซับเซตของภาษา SGML (มากกว่าที่จะเป็นแอปพลิเคชันของ SGML) และสามารถจะถูกใช้ได้ทั้งสำหรับเก็บข้อมูลและเป็นพื้นฐานสำหรับแอปพลิเคชันที่ใช้ XML เช่น XML Stylesheet Language (XSL)

เอกสาร XML (XML document) สามารถเก็บข้อมูลโดยใช้อิลิเมนต์ที่มีความหมายอย่างที่ต้องการให้มันเป็นอย่างแท้จริง แม้ว่าตอนนี้แนวความคิดนี้จะค่อนข้างใหม่แต่นี้ก็เป็นข้อได้เปรียบประการหลัก หากผู้ส่งและผู้รับ หมายความว่าผู้รับ รู้ความหมายของอิลิเมนต์นั้น เอกสาร XML สามารถถูกใช้ในการเก็บและโอนย้ายข้อมูลระหว่างแพลตฟอร์ม, ระบบปฏิบัติการและอุปกรณ์ที่เป็นอิสระจากกันอย่างแท้จริง และเนื่องจากมันเป็นแค่เท็กซ์เท่านั้น มันจึงสามารถส่งผ่านเครือข่ายชนิดต่างๆ ได้อย่างง่าย รวมถึงการส่งผ่านโปรโตคอล HTTP ในการติดต่อสื่อสารของเครือข่ายอินเทอร์เน็ตด้วย ด้วยเหตุนี้ XML จึงเป็นเวอร์ชันของ SGML ที่ใช้งานง่ายและเหมาะสำหรับเว็บ เช่นเดียวกับ SGML ตรงที่ XML ให้คุณออกแบบชุดอิลิเมนต์ของคุณเอง เมื่อต้องการอธิบายเอกสารที่เป็นเรื่องเฉพาะเหมือนกับ SGML รูปแบบที่เป็นเรื่องเฉพาะซึ่งเป็นคำจำกัดความ โปรแกรมประยุกต์ XML (หรือเรียกอีกอย่างว่า Vocabulary)

2.2 XML คืออะไร

XML เป็นภาษามาตรฐานที่ใช้โครงสร้างในการอธิบายข้อมูลซึ่งสามารถทำให้แอปพลิเคชันที่แตกต่างกันสามารถเข้าใจกันได้ โดยใช้แท็กเป็นตัวกำหนดความหมายของข้อมูลที่อยู่ในแท็กนั้นๆ ข้อมูลที่เก็บในไฟล์ XML นั้นจะอยู่ในรูป hierarchical structure เมื่อพิจารณาตัวอย่าง HTML ต่อไปนี้

```
<p><b>Mrs. Mary McGoon</b>
<br>
1404 Main Street
<br>
Anytown, NC 34829</p>
```

เมื่อนำไปเปิดด้วยเว็บเบราว์เซอร์จะแสดงผลดังนี้

Mrs. Mary McGoon

1404 Main Street

Anytown, NC 34829

จะเห็นได้ว่า HTML เป็นภาษาที่อธิบายถึงวิธีการแสดงผล (render) ของอีลิเมนต์ ไม่มีข้อมูลใดระบุถึงความหมายของข้อมูลนั้นๆ แต่ถ้าข้อมูลข้างต้นเขียนในรูปแบบของ XML จะเขียนได้เป็น

```
<address>
  <name>
    <title>Mrs.</title>
    <first-name>Mary</first-name>
    <last-name>McGoon</last-name>
  </name>
  <street>1401 Main Street</street>
  <city>Anytown</city>
  <state>MC</state>
  <zipcode>34829</zipcode>
</address>
```

จะเห็นได้ว่าเราสามารถสร้างแท็กที่แสดงถึงความหมายของข้อมูลที่อยู่ภายในแท็กนั้น และสามารถเขียนแอปพลิเคชันที่สามารถดำเนินการกับข้อมูลได้ง่ายกว่า และสามารถนำข้อมูลไปใช้งานได้หลายวัตถุประสงค์มากกว่า เช่นในการแสดงผลอาจจะนำข้อมูลเดียวกันไปแสดงผลได้หลายประเภทตามที่ต้องการได้ ซึ่งเป็นคุณสมบัติที่สำคัญของ XML ที่สามารถเก็บสารสนเทศไว้ในเอกสารได้

2.3 การสร้าง Well-Formed ให้เอกสาร XML

เนื่องจากเอกสาร XML เข้มงวดเรื่อง syntax ในเอกสารมาก ดังนั้นการที่จะสร้างเอกสาร XML นั้นต้องทำให้ถูกต้องตามกฎเกณฑ์ที่ XML ได้กำหนดขึ้น เอกสาร XML ที่สามารถนำไปใช้งานได้นั้นจะถูกรเรียกว่า Well-Formed ซึ่งมีรายละเอียดต่างๆในการสร้างดังต่อไปนี้

2.3.1 Well-Formed ของเอกสาร XML

การที่เอกสาร XML จะเป็นเอกสารที่ Well-formed นั้น ขั้นแรกเอกสารจะต้องประกอบด้วย 3 ส่วนคือ

1. Prolog ซึ่งจะประกอบด้วย การประกาศ XML(XML Declaration) เช่น
`<?xml version="1.0"?>`
2. อิลิเมนต์รูต (Root Element) เป็นอิลิเมนต์ที่อยู่นอกที่สุด ซึ่งภายในรูตนี้ก็จะเก็บอิลิเมนต์อื่นอีกด้วย นั่นคือเอกสารที่ Well-formed จะต้องประกอบด้วยอิลิเมนต์รูตหนึ่งอิลิเมนต์เท่านั้น นั่นหมายความว่า ทุกๆอิลิเมนต์ในเอกสาร XML จะต้องอยู่ในอิลิเมนต์รูต
3. Optional miscellaneous ซึ่งจะประกอบด้วย Comments, Processing Instructions

ในหัวข้อต่อไปจะเป็นรายละเอียดของการที่จะทำให้เอกสาร XML เป็นเอกสารที่ Well-formed ซึ่งจะเกี่ยวกับเรื่องต่างๆ ดังต่อไปนี้ กฎเกณฑ์(Syntax Rules) ของ XML กฎการตั้งชื่อ(Naming Rules) กฎการซ้อนทับ(Nesting Rules) เป็นต้น

2.3.2 การมาร์กอัปและข้อมูลที่เป็นตัวอักษร (Markup and Character Data)

เอกสาร XML นั้นได้สร้างขึ้นจากการมาร์กอัป (Markup) และข้อมูลที่เป็นตัวอักษร(Character data) ซึ่งการมาร์กอัปในเอกสารนั้นจะทำให้เอกสาร XML เกิดโครงสร้างขึ้น การมาร์กอัปนั้นประกอบด้วย แท็กเริ่มต้น (start tag), แท็กลงท้าย (end tag), แท็กว่าง (empty tag), Comment, Processing Instruction (PI), Document Type Declarations (DTD), ส่วนข้อมูลที่เป็นตัวอักษรนั้นก็คือ ตัวอักษรทั้งหมดที่ไม่ใช่ส่วนที่เป็นมาร์กอัป ตัวอย่างของการใช้มาร์กอัปและข้อมูลที่เป็นตัวอักษร มีดังต่อไปนี้

```
<?xml version="1.0"?>
<DOCUMENT>
  <GREETING>
    Hello from XML
  </GREETING>
  <MESSAGE>
    Welcome to the wild and wooly world of XML
  </MESSAGE>
</DOCUMENT>
```

แท็กจะต้องเริ่มด้วย “<” และลงท้ายด้วย “>” ดังนั้นแท็กในการมาร์กอัปก็คือ

`<?xml version="1.0"?>`, `<DOCUMENT>`, `<GREETING>`, `<MESSAGE>` และส่วนที่เป็นข้อมูลที่เป็นตัวอักษร คือ Hello From XML และ Welcome to the wild and wooly world of XML หัวข้อต่อไปจะเป็นการกล่าวถึงรายละเอียดของโครงสร้างเอกสาร XML ซึ่งมีเนื้อหาดังต่อไปนี้

2.3.3 การประกาศโปรล็อก (The Prolog)

Prolog จะเป็นส่วนหัวของเอกสาร XML ซึ่งก็ไม่ได้กำหนดไว้อย่างแน่นอน แต่อย่างน้อยก็ควรมีส่วนที่เป็นการประกาศ XML (XML Declaration) โดยทั่วไปแล้ว Prolog นั้นจะประกอบด้วย XML Declaration, Comment, Processing Instruction, Whitespace และ Document Type Declaration ดังตัวอย่างด้านล่างนี้

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/css" href="greeting.css"?>
<!DOCTYPE DOCUMENT [
<!ELEMENT DOCUMENT (CUSTOMER)*>
<!ELEMENT CUSTOMER (NAME,DATE,ORDERS)>
<!ELEMENT NAME (LAST_NAME,FIRST_NAME)>
<!ELEMENT LAST_NAME (#PCDATA)>
<!ELEMENT FIRST_NAME (#PCDATA)>
<!ELEMENT DATA (#PCDATA)>
<!ELEMENT ORDERS (ITEM)*>
<!ELEMENT ITEM (PRODUCT,NUMBER,PRICE)>
<!ELEMENT PRODUCT (#PCDATA)>
<!ELEMENT NUMBER (#PCDATA)>
<!ELEMENT PRICE (#PCDATA)>
]>
<!-- ***** end of Prolog ***** -->
<DOCUMENT>
...
...
```

3.3.4 การประกาศ XML (The XML Declaration)

เอกสาร XML ต้องเริ่มด้วยการประกาศ XML (XML Declaration) ซึ่งจะเป็นการแสดงว่าเอกสารนั้นถูกเขียนเป็นเอกสาร XML และควรประกาศเป็นบรรทัดแรกของเอกสาร ดังตัวอย่างด้านล่างนี้

```
<?xml version="1.0" standalone="yes" encoding="UTF-8"?>
```

การประกาศเอกสาร XML จะใช้อีลิเมนต์ `<?xml?>` ในการประกาศว่าเอกสารนี้เป็นเอกสารของ XML และใน `<?xml?>` สามารถที่จะมีแอตทริบิวต์ได้ดังต่อไปนี้

- version เป็นตัวบอกว่าใช้ XML version ใด ในที่นี้ก็จะเป็น version 1.0 ซึ่งต้องใช้แอตทริบิวต์นี้เสมอเมื่อมีการประกาศเอกสารว่าเป็นเอกสารของ XML

- encoding เป็นแอตทริบิวต์ที่ใช้อธิบายว่าจะเข้ารหัสเอกสารเป็นแบบไหน
- standalone เป็นแอตทริบิวต์ที่ใช้อธิบายว่าเรามีการอ้างอิงข้อมูลหรือใช้ข้อมูลจากภายนอกหรือไม่ ถ้ามีค่า “yes” แสดงว่าไม่ใช่ และถ้า “no” แสดงว่าใช่

2.3.5 การเขียนคอมเมนต์ (Comments)

การสร้างหมายเหตุในเอกสาร XML(XML Comment) จะเหมือนกับการ Comment ใน HTML ซึ่งแสดงถึงข้อความที่เป็นหมายเหตุพิเศษ เพื่อให้ผู้เขียนเอกสารสามารถทำความเข้าใจ และง่ายต่อการสืบค้นในภายหลัง ซึ่งจะมีในเอกสารหรือไม่ก็ได้ และไม่ถือว่าเป็นส่วนจริงของเอกสาร นั้นหมายความว่า XML Parser จะไม่สนใจ และสามารถที่จะใส่ลงในส่วนใดก็ได้ โดยข้อความหมายเหตุต้องอยู่ภายใต้เครื่องหมาย “<!--“ และ “-->“

การ Comment มีกฎดังต่อไปนี้

- ห้ามประกาศ Comment ก่อนการประกาศเอกสาร XML(XML Declaration)
- ห้ามใช้ Comment ภายในแท็ก เช่น

```
<DOCUMENT <!--Start Document -->
```
- ห้ามใช้ -- ภายใน Comment เพราะ Parser จะดูเหมือนเป็นการจบ Comment ซึ่งจะทำให้เกิด
- สามารถใช้ Comment ปิดส่วนของแท็กที่ไม่ต้องการใช้ได้

2.3.6 การใช้งาน Processing Instructions

เป็นคำสั่งที่ XML Processor จะต้องเข้าใจและสามารถทำคำสั่งนั้นได้ โดยคำสั่งจะเริ่มต้นด้วย “<?” และปิดท้ายด้วย “>” เช่นคำสั่ง <?xml-stylesheet?> เป็นคำสั่งที่ใช้เชื่อมต่อ เอกสาร XML กับ Style Sheet ของเอกสาร XML นั้น

ในส่วนที่เป็นรายละเอียดของ Prolog ได้แสดงครบแล้ว ไม่ว่าจะเป็น XML Declaration Processing Instruction และ Comment ยกเว้น DTD ซึ่งจะขออธิบายไว้ในบทต่อไป และในหัวข้อต่อไปจะกล่าวถึงรายละเอียดของโครงสร้างของเอกสาร XML คือการสร้างแท็กและอิลิเมนต์

2.3.7 แท็กและอิลิเมนต์ (Tags and Element)

การที่จะทำให้เอกสาร XML มีโครงสร้างได้นั้น สามารถทำได้โดยใช้การ Markup ซึ่งจะประกอบด้วยส่วนที่เรียกว่า อิลิเมนต์(Element) โดย XML อิลิเมนต์นั้นก็ประกอบด้วย แท็กเริ่มต้น(start tag) และ แท็กปิดท้าย(end tag) ยกเว้นอิลิเมนต์นั้นถูกกำหนดเป็นอิลิเมนต์ว่างเปล่า(empty Element) จะมีแท็กเดียว แท็กเริ่มต้น หรือเรียกอีกอย่างหนึ่งว่า Opening tag จะเริ่มต้นด้วยเครื่องหมาย “<” และลงท้ายด้วยเครื่องหมาย “>” สำหรับแท็กปิดท้ายจะเริ่มต้นด้วยเครื่องหมาย “</” และปิดท้ายด้วยเครื่องหมาย “>”

2.3.7.1 ชื่อแท็ก (Tag Names)

XML มีข้อกำหนดเกี่ยวกับการตั้งชื่อแท็กดังต่อไปนี้ โดยอักษรตัวแรกของชื่อแท็กต้องเป็นตัวอักษร underscore และ colon เท่านั้น ส่วนตัวถัดไปสามารถเป็นได้ทั้งตัวอักษร, ตัวเลข, underscore, hyphens, periods และ colon โดยต้องไม่มีการเว้นวรรค ดังตัวอย่างต่อไปนี้

```
<DOCUMENT>
<document>
<_Record>
<customer>
```

จากตัวอย่างจะเห็นได้ว่า แท็ก <DOCUMENT> และแท็ก <document> จะมีค่าไม่เท่ากัน เนื่องจาก XML Processor จะอยู่ใน Case-sensitive และจากแท็กข้างต้นนี้ จะต้องทำการปิดด้วยแท็กดังต่อไปนี้

```
</DOCUMENT>
</document>
</_Record>
</customer>
```

2.3.7.2 อิลิเมนต์ว่าง (Empty Element)

อิลิเมนต์ว่างนั้นจะประกอบด้วยแท็กเพียงแท็กเดียว ซึ่งจะไม่มีการเริ่มต้นและปิดท้าย เหมือนอย่างใน HTML เช่นแท็ก , และ
 เป็นต้น และใน XML อิลิเมนต์นั้นสามารถเขียนได้เป็น <ชื่ออิลิเมนต์ /> ดังตัวอย่างต่อไปนี้

```
<?xml version = "1.0"?>
<document>
    <greeting text = "Hello from XML" />
</document>
```

2.3.7.3 อิลิเมนต์ราก (The Root Element)

เอกสาร XML ที่ Well-formed นั้น จะต้องมีอิลิเมนต์ตัวหนึ่งที่ภายในอิลิเมนต์เก็บทุกๆ อิลิเมนต์ โดยเรียกอิลิเมนต์นี้ว่าอิลิเมนต์รากซึ่งเป็นส่วนที่สำคัญมากของเอกสาร XML โดยเฉพาะในส่วนของการพาร์ส (parse) เอกสาร XML ซึ่งจะเริ่มอิลิเมนต์รากก่อน ตัวอย่างของอิลิเมนต์รากก็เช่น อิลิเมนต์<DOCUMENT> ด้านล่าง

```
<?xml version = "1.0"?>
<DOCUMENT>
    <CUSTOMER>
        <NAME>
```

```

        <LAST_NAME></LASTNAME>
        <FIRST_NMAE></ FIRST_NMAE >
    </NAME>
    <DATE>
    </DATE>
</CUSTOMER>
</DOCUMENT>

```

2.3.7.4 แอตทริบิวต์ (Attributes)

รูปแบบของแอตทริบิวต์ของเอกสาร XML ก่อนข้างที่จะเหมือนกับของ HTML ก็จะเป็นคู่ของชื่อและค่าของแอตทริบิวต์ ซึ่งจะเป็นการกำหนดข้อมูลเพิ่มให้กับทั้งแท็กเริ่มต้น และแท็กว่าง โดยมีการกำหนดค่าของแอตทริบิวต์ในเครื่องหมาย “ ” มีรูปแบบการเขียนดังต่อไปนี้

ชื่อแอตทริบิวต์ = “ค่าแอตทริบิวต์”

ตัวอย่างด้านล่างจะเป็นตัวอย่างของการใช้แอตทริบิวต์ เมื่อเราเพิ่มแอตทริบิวต์ STATUS ให้กับอิลิเมนต์ของ <CUSTOMER>

```

<?xml version = “1.0”?>
<DOCUMENT>
    <CUSTOMER STATUS = “Good Credit”>
        <NAME>
            <FIRST_NAME>Sam</FIRST_NAME>
            <LAST_NAME>Smith</LAST_NAME>
        </NAME>
        .....
        .....
    </CUSTOMER>
</DOCUMENT>

```

เกิดคำถามขึ้นว่าเมื่อไรที่ต้องเก็บข้อมูลไว้ที่แอตทริบิวต์ และเมื่อไรที่ต้องเก็บข้อมูลไว้ที่อิลิเมนต์ การเก็บข้อมูลไว้ที่แอตทริบิวต์นั้นจะทำให้ข้อมูลไม่มีโครงสร้าง และถ้าเก็บข้อมูลไว้ที่แอตทริบิวต์มากเกินไปจะทำให้อ่านข้อมูลได้ยากเกินไป

2.3.7.4.1 ชื่อแอตทริบิวต์ (Attribute Name)

กฎการตั้งชื่อของแอตทริบิวต์นั้นจะคล้ายกับกฎการตั้งชื่อของอิลีเมนต์ คืออักษรแรกต้องเริ่มด้วยตัวอักษร, underscore, หรือ colon และอักษรตัวต่อไปอาจจะเป็นตัวอักษร, underscore, hyphens period และ colon และห้ามมีการเว้นวรรคระหว่างชื่อแอตทริบิวต์ ดังตัวอย่างด้านล่างต่อไปนี้จะเป็นการตั้งชื่อแอตทริบิวต์ที่ถูกต้อง

```
<circle origin_x= "1.0" origin_y= "20.0" radius= "10.0"/>
<image src= "image.jpg">
<pen color= "red" width= "5">
<book pages= "1231">
```

ตัวอย่างการตั้งชื่อแอตทริบิวต์ที่ผิด

```
<circle lorigin_x= "1.0" lorigin_y= "20.0" lradius= "10.0"/>
<image src name= "image.jpg">
<pen color@= "red" width@= "5">
<book pages(important)= "1231">
```

2.3.7.4.2 ค่าแอตทริบิวต์ (Attribute Values)

เนื่องจากการมาร์กอัปปกติเป็นตัวหนังสือ ดังนั้นแอตทริบิวต์ก็จะเป็นเพียงตัวหนังสือเช่นกัน และค่าของแอตทริบิวต์ปกติจะอยู่ภายใต้เครื่องหมาย “ ” ถึงแม้ว่าเราจะให้ค่าของแอตทริบิวต์เป็นตัวเลขก็ตาม

```
<circle lorigin_x= "1.0" lorigin_y= "20.0" lradius= "10.0"/>
```

เหมือนตัวอย่างนี้ XML Processor จะให้ค่าของแอตทริบิวต์เป็นสตริงเท็กซ์ ซึ่งถ้าเราต้องการใช้ข้อมูลที่เป็นตัวเลขจะต้องไปเปลี่ยนที่ภาษาที่เข้าใช้ข้อมูลของเอกสาร XML เองเช่น ภาษาจาวา เป็นต้น

สามารถใช้เครื่องหมาย ‘ ’ ในการกำหนดค่าของแอตทริบิวต์ได้เมื่อต้องการที่จะเครื่องหมาย “ ” ภายในค่าของแอตทริบิวต์อีกครั้งหนึ่ง เช่น

```
<quotation text= ‘ He said, “Not that!” ’>
```

หรือต้องการที่จะใช้เครื่องหมาย ” เพียงตัวเดียว XML Processor ก็ได้กำหนดให้สามารถใช้ ' แทนเครื่องหมาย “ ดังตัวอย่าง

```
<person height= ” 5&apos;6&apos; ”/>
```

2.3.8 การสร้างโครงสร้างเอกสารให้เป็น Well-Formed

ในการสร้างเอกสาร XML ให้เป็นเอกสารที่ Well-formed นั้นมีข้อกำหนดมากมายเกี่ยวโครงสร้างของเอกสาร XML ที่จะทำให้ตัวเอกสารนั้น Well-formed ซึ่งจะขอก้าวเป็นหัวข้อดังต่อไปนี้

1. กฎข้อแรกของโครงสร้างเอกสารที่ Well-formed คือต้องมีการประกาศว่าเป็นเอกสารของ XML (XML Declaration) ซึ่งในทางปฏิบัติเราไม่ต้องประกาศก็ได้ แต่ถ้าเราประกาศจะทำให้เอกสารของเรานั้นเป็นเอกสารที่ Well-formed
2. เอกสาร XML ที่ Well-formed จะต้องประกอบด้วยอย่างน้อยอีลิเมนต์ 1 อีลิเมนต์ ซึ่งนั่นก็คืออีลิเมนต์รูต และสำหรับทุกๆ อีลิเมนต์จะต้องถูกเก็บภายในอีลิเมนต์รูตนี้
3. สำหรับทุกๆ อีลิเมนต์ต้องประกอบด้วยแท็กเริ่มต้นและแท็กปิด ลงท้ายเสมอ ยกเว้นแท็กว่าง ซึ่งทั้งแท็กเปิดและแท็กเริ่มต้นจะต้องเหมือนกันด้วย(case-sensitive)
4. ต้องปิดแท็กว่างด้วยเครื่องหมาย /> เสมอ เนื่องจากแท็กนี้ไม่มีข้อมูลที่เป็นส่วนของ CDATA ดังนั้นจึงไม่มีการปิดแท็กด้วยแท็กปิด
5. ทำตามกฎการซ้อนทับที่ถูกต้อง (Nest Element) ซึ่งเอกสารที่ Well-formed นั้นต้องทุกๆ อีลิเมนต์ต้องอยู่ในคู่ของอีลิเมนต์ที่ถูกต้อง (Correctly Nest) ดังตัวอย่างด้านล่างเป็นเอกสารที่อยู่ในกฎการซ้อนทับที่ถูกต้อง

```
<?xml version = "1.0" encoding = "UTF-8"?>
<DOCUMENT>
  <GREETING>
    Hello From XML
  </GREETING>
  <MESSAGE>
    Welcome to the wild and wooly world of XML
  </MESSAGE>
</DOCUMENT >
```

ตัวอย่างด้านล่างนี้เป็นตัวอย่างของเอกสารที่ไม่ถูกต้องตามกฎการซ้อนทับ

```
<?xml version = "1.0" encoding = "UTF-8"?>
<DOCUMENT>
  <GREETING>
    Hello From XML
  <MESSAGE>
    Welcome to the wild and wooly world of XML
  </GREETING>
</MESSAGE>
</DOCUMENT >
```

6. ชื่อของแอตทริบิวต์ต้องไม่ซ้ำ ซึ่งจะผิดถ้าทำเช่นนี้

```
<PERSON LAST_NAME="WESTER" LAST_NAME="JONE">
```

เนื่องจาก XML เป็นกรณีที่ case-sensitive ซึ่งถ้าทำเช่นนี้จะถูก

```
<PERSON LAST_NAME="WESTER" last_name="JONE">
```

2.3.9 CDATA Sections

CDATA Sections จะเป็นส่วนของข้อมูลที่เป็น Character โดยที่ XML Processor จะไม่พาร์สข้อมูลในส่วนนี้ โดยที่ CDATA Sections จะเริ่มด้วย `<![CDATA` และปิดด้วยเครื่องหมาย `]]>` ซึ่งข้อมูลที่อยู่ภายใต้เครื่องหมายนี้ XML Processor จะทำการค้นหาข้อมูลก็จริงแต่ก็เป็นเพียงการค้นหาเครื่องหมาย `]]>` เพื่อที่จะไม่พาร์สข้อมูลในส่วนนี้ ซึ่งจะมีประโยชน์มากที่สามารถทำให้มีส่วนที่เป็นข้อมูลจริงๆ ในเอกสาร XML ที่ไม่ถูกพาร์ส

2.3.10 XML Namespaces

เนื่องจาก XML สามารถที่จะให้เรากำหนดแท็กที่ใช้งานเองได้ แต่ในปัจจุบันมีผู้สร้างเอกสาร XML มากมายซึ่งอาจจะทำให้มีผู้ที่จะกำหนดแท็กที่ซ้ำกับแท็กที่เราสร้างขึ้นก็ได้ ดังนั้นการกำหนดให้แท็กที่ใช้ซ้ำกันไม่ซ้ำกัน วิธีที่จะแก้ปัญหา คือการใช้ Namespace โดย namespace จะเป็นการรับประกันว่าถึงแม้จะมีแท็กที่ตั้งชื่อที่เหมือนกันแล้วก็ตาม แต่ทั้ง 2 แท็กนั้นจะไม่ซ้ำกัน

3.3.10.1 การสร้าง namespace (Creating a Namespace)

เพื่ออำนวยความสะดวกจะขอสร้าง namespace จากตัวอย่างต่อไปนี้

```
<library>
  <book>
    <title>
      Earthquakes for lunch
    </title>
  </book>
</library>
```

ดังนั้นเพื่อที่จะสร้าง namespace นั้นเราจะใช้ xmlns:prefix แอดทริบิวต์ในการสร้าง namespace ใหม่ ซึ่งจะต้องไม่ซ้ำกันด้วย (unique) โดย prefix ก็คือสิ่งที่จะใช้นำหน้า (prefix) อีลิเมนต์ของเรา ซึ่งเราต้องใช้พร้อมกับ URI หรือต้องอ้างโดยตรงกับ URI ของ XML Processor เมื่อกำหนดแล้วเราก็สามารถใช้ได้เป็นดังนี้

```
<library>
  <xmlns:book="http://www.amazingbooks.com/spec">
    <book:book>
      <book:title>
        Earthquakes for lunch
      </book:title>
    </book:book>
  </library>
```